

Entorno personal de aprendizaje:

CREACIÓN DE PÁGINAS WEB



Viceministerio de Tecnologías de la Información y Comunicación Director General de Inclusión Digital y TIC en la Educación

Información SEDE 1



Ministerio de
**TECNOLOGÍAS
DE LA INFORMACIÓN
Y COMUNICACIÓN**

Dirección: Complejo Santos E2 - Gral. Santos 1170 c/ Concordia Teléfono: (+595 21) 217 9000

Correo electrónico: comunicacion@mitic.gov.py Mesa de entrada: sgeneral@mitic.gov.py

Asunción – Paraguay

Curso de creación de páginas Web: Unidad 1 Protocolos de Comunicación y Arquitectura Web.

Todas las imágenes presentes en este material de apoyo educativo han sido reconocidas y, en todos los casos, son de uso libre. Todas las fuentes bibliográficas utilizadas en este material de apoyo educativo han sido citadas mediante el gestor de referencias bibliográficas Zotero, desarrollado por el Center for History and New Media de la Universidad George Mason, utilizando el estilo de la American Psychological Association Séptima Edición.



Ministerio de
**TECNOLOGÍAS
DE LA INFORMACIÓN
Y COMUNICACIÓN**

 **GOBIERNO
NACIONAL**

*Paraguay
de la gente*

Unidad 1. Conceptos Web

Contenido

1. Protocolos de Comunicación	4
1.2. Solicitudes HTTP	4
1.3. Respuestas HTTP	5
1.4. Códigos de Estado HTTP	6
2. Arquitectura de Aplicaciones Web	7
2.1. Modelo de Arquitectura	7
2.2. Ejemplos de Arquitectura de Aplicaciones Web	10
2.3. Ejemplos Específicos	10
Referencias bibliográficas	12

Unidad 1. Conceptos Web

1. Protocolos de Comunicación

Antes de sumergirnos en el mundo de los protocolos de comunicación, es importante comprender cómo, en nuestra vida cotidiana, dependemos de la tecnología para realizar tareas que van desde lo más simple hasta lo más complejo. Cada vez que navegamos por internet, vemos un video en línea, o simplemente enviamos un mensaje, estamos participando en una serie de interacciones invisibles y sofisticadas que hacen posible la transferencia de información. Esta interacción, aunque no siempre lo notamos, está mediada por diferentes protocolos de comunicación, como HTTP y HTTPS, que son esenciales para que todo funcione de manera fluida y segura.

HTTP (HyperText Transfer Protocol): Es el protocolo de comunicación utilizado en la web para la transferencia de información entre un cliente (por ejemplo, un navegador web) y un servidor. Funciona sobre el puerto 80 de TCP/IP y se utiliza para la transmisión de documentos HTML, imágenes, videos y otros tipos de datos.

HTTPS (HyperText Transfer Protocol Secure): Es una versión segura de HTTP. Utiliza SSL/TLS (Secure Sockets Layer/Transport Layer Security) para encriptar los datos transferidos entre el cliente y el servidor, asegurando la privacidad e integridad de la información. Funciona sobre el puerto 443.

1.1.Solicitudes HTTP

Una solicitud HTTP es un mensaje enviado por el cliente al servidor para solicitar un recurso o realizar una acción. Las solicitudes HTTP constan de varias partes:

Línea de solicitud: Incluye el método HTTP (como GET, POST, PUT, DELETE), la URL del recurso solicitado y la versión del protocolo HTTP.

Ejemplo:

```
GET /index.html HTTP/1.1
```

Unidad 1. Conceptos Web

Encabezados de solicitud: Contienen información adicional sobre la solicitud, como el tipo de contenido aceptado, el agente de usuario (navegador), la autenticación, etc.

Ejemplo:

```
Host: www.ejemplo.com
User-Agent: Mozilla/5.0
```

Cuerpo de la solicitud: Contiene los datos que el cliente desea enviar al servidor (solo en métodos como POST, PUT).

Ejemplo:

```
{
  "nombre": "Juan",
  "edad": 30
}
```

1.2.Respuestas HTTP

Una respuesta HTTP es el mensaje enviado por el servidor al cliente en respuesta a una solicitud HTTP. Las respuestas HTTP también constan de varias partes:

Línea de estado: Incluye la versión del protocolo HTTP, un código de estado (como 200, 404, 500) y una frase descriptiva del estado.

Ejemplo:

HTTP/1.1 200 OK

Encabezados de respuesta: Contienen información adicional sobre la respuesta, como el tipo de contenido, la fecha, el servidor, etc.

Ejemplo:

```
Content-Type: text/html
Date: Mon, 28 Jun 2024 12:00:00 GMT
```

Cuerpo de la respuesta: Contiene los datos solicitados por el cliente

Unidad 1. Conceptos Web

(como un documento HTML, JSON, etc.).

Ejemplo:

```
<html>
  <head>
    <title>Página de Ejemplo</title>
  </head>
  <body>
    <h1>Hola, Mundo!</h1>
  </body>
</html>
```

1.3.Códigos de Estado HTTP

Los códigos de estado en las respuestas HTTP indican el resultado de la solicitud. Algunos códigos de estado comunes son:

200 OK: La solicitud ha tenido éxito.

201 Created: La solicitud ha tenido éxito y se ha creado un nuevo recurso.

400 Bad Request: La solicitud es incorrecta o no puede ser procesada por el servidor.

401 Unauthorized: La autenticación es necesaria y ha fallado o aún no se ha proporcionado.

403 Forbidden: El servidor entiende la solicitud, pero se niega a autorizarla.

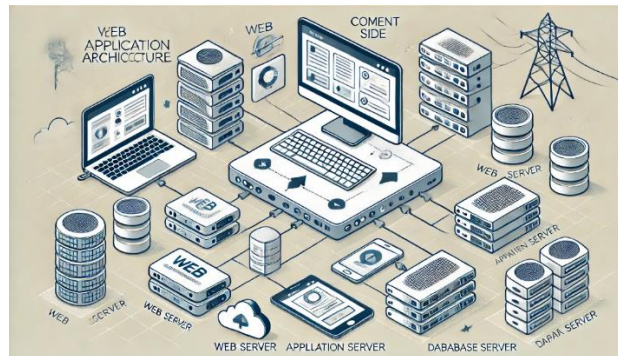
404 Not Found: El recurso solicitado no se ha encontrado.

500 Internal Server Error: Error del servidor al intentar procesar la solicitud.

2. Arquitectura de Aplicaciones Web

La arquitectura de aplicaciones web se refiere a la estructura y diseño de componentes y recursos que permiten la creación, implementación y gestión de aplicaciones web. Aquí te proporciono una guía sobre la arquitectura de aplicaciones web, sus modelos, componentes y ejemplos.

2.1. Modelo de Arquitectura



Las aplicaciones web pueden seguir varios modelos de arquitectura. Los más comunes son:

- **Cliente-Servidor**

El modelo Cliente-Servidor establece una relación donde el cliente (navegador web) solicita recursos al servidor web, que a su vez procesa estas solicitudes y devuelve los datos solicitados al cliente. Este modelo es fundamental en la arquitectura de aplicaciones web y en la comunicación distribuida en internet.

Cliente: Es la parte del sistema que solicita servicios o recursos. En el contexto de aplicaciones web, el cliente suele ser un navegador web utilizado por el usuario para acceder y solicitar información o servicios desde un servidor.

Servidor: Es la parte del sistema que responde a las solicitudes del cliente. Un servidor web es un software diseñado para procesar solicitudes HTTP (o HTTPS) provenientes de clientes, como navegadores web, y servir contenido web correspondiente, como páginas HTML, imágenes, archivos, etc.

Unidad 1. Conceptos Web

- **Multicapa (N-Tier)**

La arquitectura multicapa, también conocida como N-Tier, se refiere a un enfoque de diseño de software que organiza un sistema en capas distintas, cada una con responsabilidades específicas. Aquí te explico las capas mencionadas:

Capa de Presentación (Interfaz de usuario - navegador web): Esta capa es la que interactúa directamente con el usuario final a través de una interfaz gráfica o visual, como un navegador web en el caso de aplicaciones web. Su principal función es presentar la información de manera comprensible y permitir la interacción del usuario con el sistema.

Capa de Lógica de Negocio: En esta capa se encuentra la lógica y las reglas que gobiernan el funcionamiento del negocio o la aplicación. Aquí se procesan y manejan los datos para aplicar las reglas específicas del negocio, asegurando que las operaciones realizadas por el usuario cumplan con las políticas establecidas.

Capa de Datos: La capa de datos se encarga del almacenamiento y la gestión de los datos utilizados por la aplicación. Esto incluye bases de datos, sistemas de almacenamiento de archivos u otros repositorios donde se persisten y recuperan los datos necesarios para la aplicación. Esta capa asegura que los datos sean almacenados de manera segura y eficiente.

- **Microservicios**

Los microservicios son una arquitectura de diseño de software donde una aplicación monolítica se divide en pequeños servicios independientes y autónomos, cada uno cumpliendo una función específica y bien definida. Aquí te explico los puntos clave:

Servicios Independientes: En lugar de tener una única aplicación monolítica, los microservicios dividen funcionalidades complejas en servicios

Unidad 1. Conceptos Web

más pequeños e independientes. Cada microservicio se centra en realizar una tarea específica del negocio, como gestionar usuarios, procesar pagos o enviar correos electrónicos.

Comunicación: Los microservicios se comunican entre sí a través de interfaces bien definidas, típicamente utilizando APIs (Application Programming Interfaces). Los protocolos comunes para esta comunicación incluyen REST (Representational State Transfer) y gRPC (Google Remote Procedure Call), que permiten a los microservicios interactuar de manera eficiente y segura.

- **Serverless**

Serverless, o sin servidor, es un modelo de computación en la nube donde el proveedor de servicios en la nube (como AWS Lambda, Azure Functions, Google Cloud Functions) gestiona completamente la infraestructura subyacente necesaria para ejecutar y escalar aplicaciones. Aquí están los aspectos clave:

Funciones como Servicio (FaaS): En el contexto de serverless, las funciones como servicio (FaaS) permiten a los desarrolladores escribir pequeñas funciones de código que se ejecutan en respuesta a eventos específicos. Por ejemplo, una función puede activarse por una solicitud HTTP, una actualización en una base de datos, un archivo cargado en un almacenamiento, entre otros eventos. La plataforma cloud asigna recursos automáticamente para ejecutar estas funciones sin que el desarrollador tenga que preocuparse por la infraestructura subyacente.

Backend as a Service (BaaS): Además de las funciones como servicio, el concepto de serverless también incluye servicios backend gestionados (BaaS), que ofrecen funcionalidades como autenticación de usuarios, bases de datos, almacenamiento de archivos, entre otros servicios comúnmente necesarios para aplicaciones web o móviles. Estos servicios son gestionados por el proveedor de la nube y se consumen mediante APIs, liberando al desarrollador de la tarea de administrar la infraestructura y permitiéndoles enfocarse en la lógica de negocio.

Unidad 1. Conceptos Web
de la aplicación.

2.2.Ejemplos de Arquitectura de Aplicaciones Web

- **Aplicación Monolítica**

Descripción: Todo el código (UI, lógica de negocio, acceso a datos) está en un solo proyecto.

Ejemplo: Una aplicación e-commerce simple desarrollada con Django.

- **Aplicación Multicapa (N-Tier)**

Descripción: Separación de la capa de presentación, lógica de negocio y datos.

Ejemplo: Aplicación bancaria donde el frontend está en Angular, el backend en Spring Boot, y la base de datos en PostgreSQL.

- **Microservicios**

Descripción: Aplicación dividida en múltiples servicios pequeños e independientes.

Ejemplo: Netflix, donde cada funcionalidad (recomendaciones, reproducción, gestión de usuarios) es un microservicio separado.

- **Aplicación Serverless**

Descripción: Funcionalidades gestionadas y ejecutadas por proveedores cloud.

Ejemplo: Aplicación de procesamiento de imágenes usando AWS Lambda para procesamiento y S3 para almacenamiento.

2.3.Ejemplos Específicos

- **Netflix (Microservicios)**

Unidad 1. Conceptos Web

Descripción: Utiliza una arquitectura de microservicios para escalar y gestionar diferentes funcionalidades independientemente.

- **Amazon (Multicapa)**

Descripción: Infraestructura basada en una arquitectura multicapa para manejar millones de transacciones diarias, integrando servicios front-end, lógica de negocio compleja y bases de datos robustas.

- **Slack (Cliente-Servidor)**

Descripción: Interfaz de usuario dinámica y rica en JavaScript (React) comunicándose con un backend en tiempo real (Node.js y WebSocket).

- **Instagram (Monolítica en Origen, luego Microservicios)**

Descripción: Comenzó como una aplicación monolítica en Django, evolucionando hacia una arquitectura de microservicios para manejar su crecimiento.

Unidad 1. Conceptos Web

Referencias bibliográficas

CampusMVP. (s. f.). Qué es un desarrollador web: Front-end, back-end y full stack. ¿Quién es quién? - campusMVP.es. campusMVP.es.
<https://www.campusmvp.es/recursos/post/que-es-un-desarrollador-web-front-end-back-end-full-stack.aspx>

¿Cuál es la diferencia entre un desarrollador Front-End, Back-End y Full Stack? (s. f.). Instituto Idat. <https://www.idat.edu.pe/blog/cual-es-la-diferencia-entre-un-desarrollador-front-end-back-end-y-full-stack>

Hernández, R. (2022, 19 agosto). Diferencia entre Front-end, Back-end y Full-stack: nociones básicas para reclutadores de Talento IT. HRTRENDS.
<https://empresas.infoempleo.com/hrtrends/diferencia-front-end-back-end-y-full-stack-nociones-basicas-para-reclutadores/>

KeepCoding, R. (2023, 8 junio). Front End, Back End y Full Stack, 3 tipos de developer codiciados. KeepCoding Bootcamps.
<https://keepcoding.io/blog/frontend-backend-y-full-stack-3-tipos-developer/>

Sys, A. (2022). Descubre qué tipo de desarrollador eres: Front End, Backend o Full Stack. CodersLink. <https://coderslink.com/talento/blog/descubre-que-tipo-de-desarrollador-eres-front-end-backend-o-full-stack/>